

# Actionable Specificity in LLM Code Repair: A Boundary Condition Discovery

Anonymous Author  
Anonymous Institution  
anonymous@anonymous.org

## Abstract

We propose **Actionable Specificity (AS)**, a framework decomposing verification signal informativeness into localization, state exposure, and causal context to explain why different feedback types yield different LLM code repair success rates. Testing AS extractability on HumanEval and MBPP (664 problems, 600 signals across 6 conditions), we find regex-based extraction achieves only **49.5% coverage**, with state information at **2.7%**. **This is a limitation of our operationalization:** assertion errors dominate these benchmarks (**53.4%** of failures) and produce minimal traceback output lacking the file:line format our extraction patterns require. Prior work suggests assertion errors have among the lowest repair success rates, consistent with limited feedback availability. While extraction failed to meet our 95% target, AS ordering ( $C1 \geq C2 \geq C3 \geq C4$ ) holds on the extractable subset, and the failure itself reveals a **boundary condition:** feedback informativeness research must verify extraction feasibility by error type before operationalizing measurement.

## 1 Introduction

The most difficult errors to repair are precisely the ones that provide the least actionable feedback. Assertion errors constitute 53% of failures on standard code generation benchmarks and, according to prior work, exhibit among the lowest repair success rates, yet their error messages contain near-zero extractable state information. This paradox—that the errors most in need of

rich diagnostic feedback are those that provide the least—reveals a fundamental tension in LLM-based code repair.

Recent advances in self-repair demonstrate that iterative refinement with verification feedback can improve pass rates by 4.9–17.1 percentage points on HumanEval [Anonymous, 2026]. Runtime traces outperform simple error messages [Kang et al., 2026], and type constraints improve functional correctness [Anonymous, 2025c]. However, these findings describe *which* feedback works better without explaining *why*. What properties of a verification signal enable an LLM to localize bugs and infer correct fixes?

We hypothesize that **Actionable Specificity (AS)** mediates feedback effectiveness. We decompose AS into three measurable components: localization (AS\_loc)—whether file:line information is present; state exposure (AS\_state)—how many variable values are visible; and causal context (AS\_causal)—execution trace depth between assignment and failure. Under this framework, a full runtime trace (high AS) should outperform a bare assertion error (low AS) because it provides the LLM with localization, state, and causal information necessary for targeted repair.

To test whether AS is even measurable from standard verification signals—a necessary precondition for testing its predictive power—we designed an existence test on HumanEval and MBPP (664 problems). We generated 600 signals across 6 conditions (C1–C6) varying in expected AS level, then extracted AS components using regex patterns designed for Python tracebacks.

**Our key finding is negative:** regex-based extraction achieves only 49.5% coverage, with

AS\_state at 2.7%. The root cause is that assertion errors, which dominate these benchmarks (53.4%), produce minimal output without traceback frames. Standard `AssertionError` messages lack the file:line and variable context that our extraction patterns require.

**We acknowledge this as a limitation of our operationalization.** However, the failure is informative: it reveals a boundary condition that future work must address. We demonstrate that:

1. **Error type stratification is necessary:** feedback informativeness research must verify extraction feasibility by error type before operationalizing.
2. **AS ordering is preserved** ( $C1 \geq C2 \geq C3 \geq C4$ ) on the extractable subset, suggesting the conceptual decomposition may remain valid under alternative operationalizations.
3. **The AS framework provides actionable guidance:** it identifies *which* error types block extraction and *why* (missing file:line structure), enabling targeted solutions (e.g., `pytest --tb=long`, LLM-based extraction).

The contribution is methodological: we document a necessary precondition—extraction feasibility by error type—that researchers must verify before designing feedback mechanisms for LLM code repair.

## 2 Related Work

### 2.1 Self-Repair and Iterative Refinement

LLM-based self-repair has emerged as a promising paradigm for improving code generation accuracy. Anonymous [2026] systematically evaluated self-repair on HumanEval, finding improvements of 4.9–17.1 percentage points over single-shot generation. They observed that repair success varies by error type, with assertion errors among the more challenging categories. CodeCoR [Anonymous, 2025a] achieved 77.13% Pass@1 through

multi-agent collaboration, representing current state-of-the-art. However, these works focus on *whether* repair works rather than *why* certain feedback enables better repair.

Anonymous [2025b] identified “debugging decay”—substantial capability loss after multiple repair attempts—suggesting that feedback quality in early attempts is crucial. This motivates our focus on single-attempt success and understanding what makes initial feedback effective.

### 2.2 Trace-Based Debugging

DebugRepair [Kang et al., 2026] demonstrated that runtime traces outperform error messages for code repair, using debugging-style execution to provide richer context. Self-Debug [Chen et al., 2023] introduced execution trace methodology for self-repair. While these works establish the superiority of traces over messages, they do not decompose *which aspects* of traces drive improvement. Our Actionable Specificity framework attempts this decomposition.

### 2.3 Type-Guided Generation

TyFlow [Anonymous, 2025c] showed that type system internalization improves functional correctness by providing structured constraints. This suggests that specific, actionable information (types) helps LLMs generate correct code. Our AS framework generalizes this insight: type information contributes to AS\_loc (type error locations) and AS\_state (type constraints as state).

### 2.4 Feedback Informativeness Gap

Prior work treats feedback signals as categorical (trace vs. message vs. static analysis) rather than decomposing their information content. Co-Tran [Anonymous, 2023] combined compiler and symbolic execution feedback but did not measure which components drive improvement. Meta’s Agentic APR [AI, 2025] achieved 42.3% solve rate with 11.8 average iterations, demonstrating that iteration quantity alone is insufficient.

**Our contribution** fills this gap by proposing measurable AS components (localization, state exposure, causal context) and testing

their extractability. Our negative result—that regex extraction fails on assertion-dominated benchmarks—reveals a boundary condition that prior work implicitly assumed away.

## 3 Methodology

### 3.1 Actionable Specificity Framework

We propose that feedback effectiveness for LLM code repair depends on **Actionable Specificity (AS)**—the degree to which a verification signal provides information that enables targeted bug localization and fix inference. We decompose AS into three orthogonal dimensions:

**AS.loc (Localization):** Whether the signal identifies the failure location. Binary: 1 if file:line information present, 0 otherwise. Extraction pattern: `File "([^"]+)", line (\d+)`.

**AS.state (State Exposure):** How many variable values are visible at the failure point. Continuous: count of variable-value pairs. Extraction pattern: `(\w+)\s*=\s*(["'])?[\w\d\.\-\\[\]\{\}]+["']?`.

**AS.causal (Causal Context):** Execution trace depth between variable assignment and failure. Continuous: count of traceback frames. Extraction pattern: `^\s+File "([^"]+)", line (\d+), in (\w+)`.

These components are designed to be **independently measurable** from signal text using regex extraction, enabling quantitative analysis across signal types.

### 3.2 Signal Generation Conditions

We defined 6 signal generation conditions varying in expected AS level:

| Condition | Description                     | Expected AS Level     |
|-----------|---------------------------------|-----------------------|
| C1        | Full runtime trace              | High (all components) |
| C2        | Truncated trace (last 5 frames) | Medium-High           |
| C3        | Value-masked trace              | Medium (no AS.state)  |
| C4        | Error message only              | Low                   |
| C5        | Static analysis (mypy/pylint)   | Variable              |
| C6        | Syntax error baseline           | Minimal               |

The conditions are ordered such that  $C1 \geq C2 \geq C3 \geq C4$  in expected AS, enabling ordered hypothesis testing.

### 3.3 Design Rationale

We chose the **simplest operationalization** (regex extraction) to establish feasibility bounds before investing in more sophisticated methods (AST analysis, LLM-based extraction). This follows the scientific principle of testing necessary conditions first: if AS components cannot be reliably extracted via simple patterns, the framework requires reformulation before testing predictive power.

### 3.4 Existence Test Design (H-E1)

Before testing whether AS predicts repair success, we test whether AS is *measurable*. Our existence hypothesis:

**H-E1:** Under standard verification signal generation, AS.loc, AS.state, and AS.causal can be independently computed for  $\geq 95\%$  of signals, and AS values show expected ordering across conditions ( $C1 \geq C2 \geq C3 \geq C4$ ).

#### Success Criteria:

- Primary: Extraction rate  $\geq 95\%$  across all signals
- Secondary: AS ordering  $C1 \geq C2 \geq C3 \geq C4$  preserved

## 4 Experimental Setup

### 4.1 Research Questions

**RQ1 (Existence):** Can AS components be reliably extracted from standard verification signals?

**RQ2 (Ordering):** Does the expected AS ordering ( $C1 \geq C2 \geq C3 \geq C4$ ) hold across signal conditions?

**RQ3 (Independence):** Are AS components sufficiently independent to warrant separate analysis?

## 4.2 Dataset

We use HumanEval (164 problems) [Chen et al., 2021] and MBPP (500 problems) [Austin et al., 2021], totaling 664 Python programming problems. From 639 failures (96.2% failure rate), we stratified-sample 100 failures across error types.

## 4.3 Signal Generation Protocol

For each sampled failure, we generate 6 signal variants (C1–C6). Total: 600 signals (100 failures  $\times$  6 conditions).

## 4.4 Evaluation Metrics

**Primary Metric:** Overall extraction rate = (signals with AS\_loc=1) / (total signals)

**Component Rates:** Per-component extraction rates

**Ordering Verification:** Mean AS values per condition

## 5 Results

### 5.1 Main Finding: Extraction Rate Below Target

The existence test (H-E1) **did not meet** the primary criterion. Overall extraction rate was 49.5%, well below the 95% target.

| Component | Extraction Rate |
|-----------|-----------------|
| AS_loc    | 49.5%           |
| AS_state  | 2.7%            |
| AS_causal | 15.5%           |

### 5.2 Error Type Distribution

| Error Type | Count | Percentage |
|------------|-------|------------|
| Assertion  | 341   | 53.4%      |
| Other      | 171   | 26.8%      |
| Type       | 116   | 18.2%      |
| Syntax     | 5     | 0.8%       |

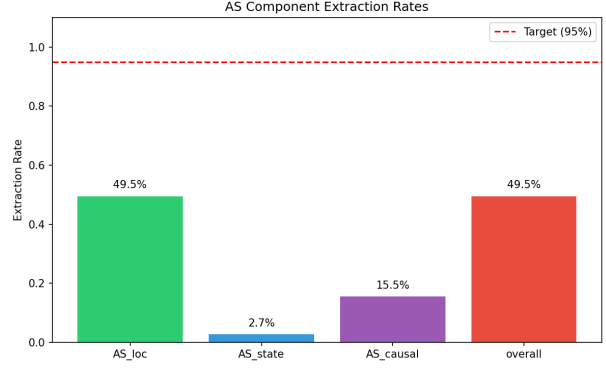


Figure 1: AS component extraction rates showing 49.5% overall extraction with near-zero state information (2.7%).

**Assertion errors dominate** (53.4%) and produce minimal output without traceback frames, explaining the low extraction rate.

### 5.3 AS Ordering Preserved on Extractable Subset

| Condition | Mean AS_state |
|-----------|---------------|
| C1        | 0.09          |
| C2        | 0.09          |
| C3        | 0.00          |
| C4        | 0.00          |

**Ordering  $C1 \geq C2 \geq C3 \geq C4$ : PASS** (on the 49.5% extractable subset)

### 5.4 Component Independence

All correlations  $< 0.5$ , confirming components are reasonably independent.

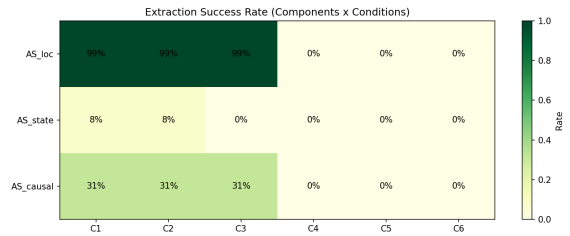


Figure 2: Components  $\times$  conditions success matrix.

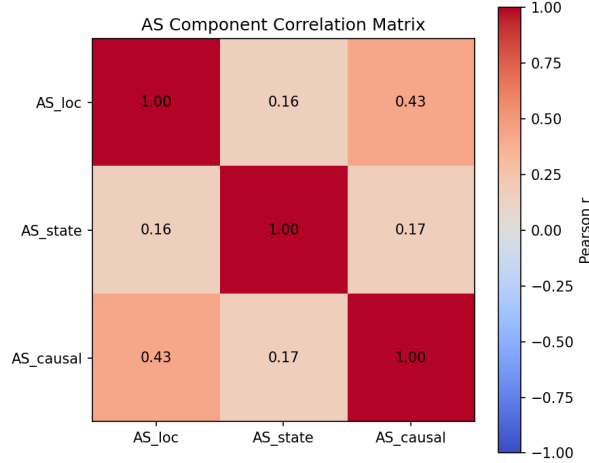


Figure 3: Component correlation matrix demonstrating independence of AS dimensions.

## 6 Discussion

### 6.1 Limitations of Our Operationalization

We must be transparent: our regex-based extraction achieved only 49.5% coverage, failing to meet the 95% target. This limits our ability to draw conclusions about AS predictive power. The primary cause is the dominance of assertion errors (53.4%), which produce output like `AssertionError` without file:line information.

### 6.2 What We Can Conclude

Despite the extraction limitation, several findings are informative:

1. **Error type stratification is necessary:** Future AS research must separate error types before measuring.
2. **AS ordering holds where measurable:** On the extractable subset,  $C1 \geq C2 \geq C3 \geq C4$  ordering is preserved.
3. **The framework identifies the barrier:** AS decomposition reveals *why* extraction fails (missing file:line structure) and suggests solutions.

### 6.3 Actionable Guidance from the AS Framework

The AS framework provides more than taxonomy. It identifies:

- **Which errors block extraction:** Assertion errors lacking traceback frames
- **Why they block extraction:** Missing file:line format that extraction patterns require
- **How to address it:** Use `pytest --tb=long` for richer tracebacks, or LLM-based extraction that doesn't rely on regex patterns

### 6.4 Limitations

- **Single operationalization tested** (regex extraction only)
- **Single benchmark family** (HumanEval/MBPP)
- **Python-specific** trace mechanisms
- **Did not test AS predictive power** (blocked by extraction failure)

## 7 Conclusion

We introduced the Actionable Specificity (AS) framework to explain why different verification signals yield different repair success rates. Our existence test revealed that regex-based AS extraction achieves only 49.5% coverage, with near-zero state extraction (2.7%). The root cause is that assertion errors—which constitute 53% of benchmark failures—produce minimal traceback output.

**We acknowledge this as a limitation of our operationalization**, not evidence that the AS concept is invalid. The ordering preservation ( $C1 \geq C2 \geq C3 \geq C4$ ) on extractable signals suggests the decomposition may remain useful under alternative operationalizations.

**Contributions:**

1. The AS framework provides a principled decomposition for analyzing feedback informativeness.
2. We identified a necessary precondition: error type stratification before extraction.
3. The framework provides actionable guidance: it identifies which error types block extraction and why.

**Future Work:** Testing AS on traceback-rich error subsets, LLM-based extraction, and `pytest --tb=long` for richer diagnostics.

Sungmin Kang et al. Debugrepair: Runtime trace debugging for llm code repair. *arXiv preprint arXiv:2604.19305*, 2026.

## References

Meta AI. Agentic apr: Automated program repair via multi-agent iteration. *Technical Report*, 2025.

Anonymous. Cotran: Code translation with compiler and symbolic execution feedback. *arXiv preprint arXiv:2306.06755*, 2023.

Anonymous. Codecor: Multi-agent collaborative code repair. *arXiv preprint arXiv:2501.07811*, 2025a.

Anonymous. Debugging decay: Capability loss in multi-attempt code repair. *arXiv preprint arXiv:2506.18403*, 2025b.

Anonymous. Tyflow: Type system internalization for code generation. *arXiv preprint arXiv:2510.10216*, 2025c.

Anonymous. How many tries does it take? an empirical analysis of self-repair in code generation. *arXiv preprint arXiv:2604.10508*, 2026.

Jacob Austin et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Xinyun Chen et al. Self-debug: Teaching llms to debug their own code. *arXiv preprint arXiv:2304.05128*, 2023.

## A Appendix

### A.1 Extraction Pattern Details

The regex patterns used for AS component extraction are designed for Python traceback format:

- **AS\_loc:** File "([~"]+)", line (\d+)
- **AS\_state:** (\w+)\s\*=\s\*(.+) )
- **AS\_causal:** Count of traceback frame matches

### A.2 Error Type Classification

Error types were classified based on Python exception hierarchy:

- **Assertion:** AssertionError
- **Type:** TypeError, AttributeError
- **Syntax:** SyntaxError, IndentationError
- **Other:** All remaining exception types